

Proper Way to Setup Database with Backend

Step-by-step guide for integrating PostgreSQL with Express using Sequelize

■ Install Dependencies

```
npm i express dotenv cors pg pg-hstore sequelize
npm i --save-dev sequelize-cli
```

■ Initialize Sequelize

```
npx sequelize-cli init
```

■ Setup Config with .env

```
import { config } from "dotenv";
config();

export default {
  development: {
    username: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
    database: process.env.DB_NAME,
    host: process.env.DB_HOST,
    dialect: "postgres",
  },
};
```

■ Environment Variables

```
PORT=5000
DB_USER=usman_nadiry
DB_PASSWORD=nadiry_root
DB_NAME=chatbot_db
DB_HOST=localhost
```

■ Create connectdb.js

```
import { Sequelize } from "sequelize";
import { dbConfig } from './config.db.js';

const { name, user, password, host, dialect } = dbConfig;

const sequelize = new Sequelize(name, user, password, {
  host,
  dialect
});
```

```
export const connectDB = async () => {
  try {
    sequelize.authenticate();
    console.log('sequelize db connected');
  } catch (error) {
    console.log('error connecting sequelize db.', error);
  }
};
```

■ Setup app.js

```
import express from "express";
import { config } from 'dotenv';
import cors from "cors";
import { connectDB } from "../config/connectdb.js";

config();

const PORT = process.env.PORT;
const app = express();

app.use(express.json());
app.use(cors());

app.listen(PORT, () => {
  console.log("server running on port 5000");
  try {
    connectDB();
  } catch (error) {
    console.log('db connection failed', error);
  }
});
```

■■ Create Database

```
npx sequelize-cli db:create
```

■ Verify Database

```
\l
```

■ Note

If you want to create a new database with a new user and password, run the following SQL command:

```
CREATE USER usman_nadiry WITH PASSWORD 'nadiry_root' CREATEDB;
```

Put this name and password in config.js before running:

```
npx sequelize-cli db:create
```